CI/CD for DevOps Engineers

Complete Study Guide

Jenkins | GitHub Actions | Azure DevOps Pipelines

Pipeline Stages | *Templates* | *Approval Gates* | *Best Practices*

Manual Tester → DevOps Engineer Transition

TABLE OF CONTENTS

1. PART 1: CI/CD FUNDAMENTALS
2. PART 2: JENKINS
3. PART 3: GITHUB ACTIONS
4. PART 4: AZURE DEVOPS PIPELINES
5. PART 5: ADVANCED TOPICS
6. PART 6: INTERVIEW PREPARATION
7. PART 7: HANDS-ON PRACTICE & PROJECTS
8. PART 8: COMMON MISTAKES & TROUBLESHOOTING
9. PART 9: QUICK REVISION GUIDE

PART 1
CI/CD FUNDAMENTALS

1. What is CI/CD?

1.1 The Problem CI/CD Solves

Traditional software development (before CI/CD):

- Developers work in isolation for weeks or months
- Code is integrated all at once before release (integration hell)
- Testing happens at the end (bugs discovered late)
- Manual deployments (error-prone, slow, stressful)
- Releases every few months (risky, big-bang changes)
- No confidence in what's deployed

With CI/CD:

- Developers integrate code multiple times daily
- Automated tests run on every commit (bugs found immediately)
- Deployments are automated and frequent (low-risk, small changes)
- Releases can happen multiple times daily
- High confidence — every change is tested and validated

1.2 CI vs CD — Clear Definitions

CI (Continuous Integration)

Continuous Integration is the practice of automatically building and testing code every time a developer pushes changes to version control.

▸ **Key Principles**

- **Frequent Commits** — Developers push code at least daily, often multiple times
- **Automated Build** — Every commit triggers an automatic build
- **Automated Tests** — Unit tests run automatically — must pass before merge
- **Quick Feedback** — Developers know within minutes if their code broke something
- **Single Source Repository** — All code in one place (Git), everyone works from latest

CD — Two Different Meanings

▸ **Continuous Delivery**

Code is ALWAYS in a deployable state. Every change that passes automated tests CAN be deployed to production, but requires manual approval.

- Automated up to staging/pre-prod
- Manual approval before production deployment

- Deploy button is one click away
- Used when: business wants control over release timing, regulated industries

▸ **Continuous Deployment**

Every change that passes automated tests is AUTOMATICALLY deployed to production. No human intervention.

- Fully automated from commit to production
- No manual approval gates (except for rare exceptions)
- Deploys to production multiple times per day
- Used when: mature CI/CD, comprehensive tests, feature flags in place

Stage	Traditional	Continuous Delivery	Continuous Deployment
Developer Commits	Yes	Yes	Yes
Automated Build	Yes	Yes	Yes
Automated Tests	Yes	Yes	Yes
Deploy to Staging	Manual	Automatic	Automatic
Deploy to Production	Manual	Manual (one-click)	Automatic
Release Frequency	Weekly/Monthly	Daily/Weekly	Multiple times daily

📌 **NOTE:** Most companies use Continuous Delivery, not full Continuous Deployment. They want some human oversight before production release.

---

2. CI/CD Pipeline Stages

2.1 Standard Pipeline Flow

A typical CI/CD pipeline has these stages. Not every pipeline needs all of them — adapt to your needs.

10. Source / Trigger — Code commit triggers pipeline
11. Build — Compile code, resolve dependencies, create artifact
12. Unit Test — Fast tests of individual functions
13. Code Quality — Linting, code coverage, complexity analysis
14. Security Scan — Static analysis for vulnerabilities
15. Package — Create deployable artifact (JAR, Docker image, ZIP)
16. Deploy to Staging — Deploy to non-production environment
17. Integration Test — Test how services work together
18. Acceptance Test — Automated E2E tests simulating user behavior
19. Deploy to Production — Ship to production (manual or automatic)
20. Smoke Test — Verify core functionality after production deploy

21. Monitor — Watch for errors, performance issues

Stage Details

▸ 1. Source / Trigger

What kicks off the pipeline?

- **Push Trigger** — Developer pushes to Git → pipeline starts immediately
- **Pull Request Trigger** — PR created/updated → run tests before allowing merge
- **Scheduled Trigger** — Run daily at 2 AM (for nightly builds, reports)
- **Manual Trigger** — Click button to start pipeline on-demand

▸ 2. Build Stage

Compile code and create runnable artifact.

Language	Build Command	Output
Java	mvn clean package	Creates JAR or WAR file
Node.js	npm install && npm run build	Creates dist/ folder with bundled JS
.NET	dotnet build	Creates DLL or EXE
Python	pip install -r requirements.txt	Install dependencies (no compilation)
Go	go build -o app	Creates binary executable
Docker	docker build -t myapp:${BUILD_ID}	Creates Docker image

▸ 3. Unit Test Stage

Run fast, isolated tests. Goal: under 10 minutes for entire test suite.

- **Test Frameworks** — JUnit (Java), Jest (Node.js), pytest (Python), NUnit (.NET)
- **Coverage Target** — Aim for 70-80% code coverage minimum
- **Fail Fast** — If any test fails, pipeline stops — no point continuing
- **Test Reports** — Generate JUnit XML for reporting in CI/CD UI

```
# Java example
mvn test                          # Run unit tests
mvn jacoco:report                 # Generate coverage report
```

```
# Node.js example
npm test -- --coverage            # Run tests with coverage
```

```
# Python example
pytest --cov=app --cov-report=xml # Run tests with coverage
```

▸ 4. Code Quality Stage

Analyze code for issues before they reach production.

Tool	What It Does	Checks For

SonarQube	Comprehensive code quality platform	Bugs, code smells, security hotspots, coverage
ESLint	JavaScript/TypeScript linter	Code style, common errors
Pylint / Flake8	Python linters	PEP 8 violations, code quality
Checkstyle	Java code style checker	Formatting, naming conventions

```
# SonarQube scan example
sonar-scanner \
  -Dsonar.projectKey=myapp \
  -Dsonar.sources=. \
  -Dsonar.host.url=http://sonarqube:9000 \
  -Dsonar.login=${SONAR_TOKEN}
```

▸ 5. Security Scan (SAST)

Static Application Security Testing — scan source code for vulnerabilities.

Tool	Purpose
OWASP Dependency Check	Checks for known vulnerabilities in dependencies
Snyk	Finds and fixes vulnerabilities in dependencies
Trivy	Scans Docker images for CVEs
GitLeaks / git-secrets	Prevents secrets (passwords, API keys) from being committed
Checkmarx / Fortify	Enterprise SAST tools for source code analysis

▸ 6. Package Stage

Create deployable artifact.

Artifact Type	How to Create	Use Case
Docker Image	docker build -t app:$VERSION	Most common for microservices
JAR/WAR	mvn package	Java applications
NPM Package	npm pack	Node.js libraries
Helm Chart	helm package ./chart	Kubernetes deployment package
ZIP/TAR	tar -czf app.tar.gz dist/	Static files, serverless functions

▸ 7-9. Deploy & Test Stages

Deploy to staging, run integration and E2E tests, then deploy to production if all pass.

```
# Typical deployment flow
1. Deploy to Staging environment
2. Run smoke tests (is app responding?)
```

```
3. Run integration tests (do services communicate correctly?)
4. Run E2E tests (simulate real user workflows)
5. If all pass: Deploy to Production (or wait for approval)
6. Run smoke tests on Production
7. Monitor for errors for 15 minutes
8. If errors detected: Automatic rollback
```

▸ **10-12. Post-Deploy & Monitoring**

Deployment isn't done when code is live — you must verify it works!

- **Smoke Tests** — Quick sanity checks — is homepage loading? Is API responding?
- **Health Checks** — Hit /health endpoint, verify 200 OK response
- **Monitoring** — Watch error rates, latency, memory usage for anomalies
- **Alerts** — If error rate spikes or latency increases, alert on-call engineer

💡 **TIP:** A pipeline that doesn't include automated tests is just automated deployment, not CI/CD.

---

3. Pipeline Best Practices

3.1 Pipeline Design Principles

- **Fail Fast** — Run fastest tests first. If unit tests fail, don't waste time on integration tests.
- **Build Once, Deploy Many** — Create artifact once, deploy same artifact to staging then production.
- **Immutable Artifacts** — Never rebuild for production — use exact artifact tested in staging.
- **Environment Parity** — Staging should be as close to production as possible (same OS, DB version, config).
- **Idempotent Pipelines** — Running pipeline multiple times should be safe (no accidental double-deploys).
- **Atomic Deployments** — Deployment succeeds completely or fails completely (no half-deployed state).
- **Blue-Green / Canary** — Deploy new version alongside old, switch traffic gradually, easy rollback.

3.2 Pipeline Performance

Technique	How	Impact
Parallel Execution	Run unit tests and code quality scans in parallel	Reduce pipeline time by 50%
Caching Dependencies	Cache npm_modules, Maven .m2, pip cache	Build time: 10 min → 2 min
Docker Layer Caching	Reuse unchanged Docker layers	Image build: 8 min → 1 min
Incremental Builds	Only rebuild changed modules	Large projects: save 70% time

Smart Test Selection	Only run tests affected by code changes	Test time: 20 min → 3 min

3.3 Pipeline Security

- **Secrets Management —** NEVER hardcode passwords. Use Azure Key Vault, AWS Secrets Manager, or CI/CD platform's secrets.
- **Least Privilege —** Pipeline should have minimum permissions needed. Separate credentials for staging vs production.
- **Dependency Scanning —** Check third-party libraries for known vulnerabilities before deploying.
- **Image Scanning —** Scan Docker images for CVEs. Block deployment if critical vulnerabilities found.
- **Access Control** — Only authorized people can trigger production deployments.
- **Audit Logs —** Track who deployed what, when. Essential for compliance and troubleshooting.

⚠️ **WARNING:** Common mistake: Storing production database password in pipeline YAML file committed to Git. This is a major security breach!

PART 2

JENKINS

4. Jenkins Architecture

4.1 What is Jenkins?

Jenkins is an open-source automation server. It's the most popular CI/CD tool — used by millions of projects worldwide. Jenkins runs pipelines, called 'Jobs', which can build, test, and deploy applications.

Why Jenkins is Popular

- **Free & Open Source** — No licensing costs, huge community
- **1800+ Plugins** — Integrates with everything — Git, Docker, Kubernetes, AWS, Azure, Slack
- **Flexibility** — Can build anything — Java, Python, Node.js, .NET, mobile apps
- **Mature & Stable** — Been around since 2011, battle-tested in production
- **Pipeline as Code** — Define pipelines in Jenkinsfile (version controlled)

4.2 Jenkins Architecture

Components

▸ **Jenkins Master (Controller)**

The brain of Jenkins. Responsibilities:

- Schedules and orchestrates pipeline jobs
- Monitors agents and assigns work to them
- Serves the Jenkins web UI (port 8080)
- Stores configuration, build history, logs
- Should NOT run actual builds (delegate to agents)

▸ **Jenkins Agent (formerly Slave)**

Worker nodes that execute pipeline steps. Benefits of agents:

- Scale: Run many builds in parallel
- Isolation: Different environments (Linux, Windows, Docker)
- Performance: Master doesn't get overloaded
- Security: Builds run in isolated environments

```
# Agent types:
- Permanent Agents: Always connected (dedicated VMs)
- Cloud Agents: Spin up on-demand (Docker, Kubernetes, AWS)
- SSH Agents: Master connects via SSH
```

How Jenkins Works — Build Flow

22. Developer pushes code to Git

23. Git webhook triggers Jenkins (or Jenkins polls Git)
24. Jenkins Master receives trigger
25. Master reads Jenkinsfile from repository
26. Master creates a build job and assigns to an available agent
27. Agent checks out code from Git
28. Agent executes pipeline stages (build, test, deploy)
29. Agent sends logs back to Master
30. Master stores results and shows in UI
31. Jenkins sends notification (Slack, email) based on result

Jenkins Architecture Diagram (Conceptual)

```
                        ┌──────────────┐
Developer ──────────>│     Git      │
(push code)             └──────────────┘
                                │ webhook
                                ▼
                        ┌──────────────┐
                        │   Jenkins    │
                        │   Master     │◄──── Admin configures
                        │  (port 8080) │
                        └──────────────┘
                                │ assigns work
              ┌─────────────────┼─────────────────┐
              ▼                 ▼                 ▼

        ┌──────────┐      ┌──────────┐      ┌──────────┐
        │ Agent 1  │      │ Agent 2  │      │ Agent 3  │
        │ (Linux)  │      │(Windows) │      │ (Docker) │
        └──────────┘      └──────────┘      └──────────┘
```

4.3 Jenkinsfile

Jenkinsfile is a text file that defines your pipeline as code. It lives in your Git repository alongside your application code. Two syntaxes:

Declarative Pipeline (Recommended)

Simpler, more structured. Best for 90% of use cases.

```
pipeline {
    agent any  // Run on any available agent

    stages {
        stage('Build') {
            steps {
                echo 'Building...'
                sh 'mvn clean package'
            }
        }
```

```
        stage('Test') {
            steps {
                echo 'Testing...'
                sh 'mvn test'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploying...'
                sh './deploy.sh'
            }
        }
    }

    post {
        success {
            echo 'Pipeline succeeded!'
        }
        failure {
            echo 'Pipeline failed!'
        }
    }
}
```

Declarative Pipeline Structure

Directive	Purpose
pipeline { }	Top-level block — every Declarative pipeline starts with this
agent	Where to run (any, none, label:'linux', docker:'node:16')
environment	Define environment variables for entire pipeline
stages	Container for all stages
stage	Logical grouping of steps (Build, Test, Deploy)
steps	Actual commands to run (sh, bat, echo, script)
post	Actions after pipeline completes (success, failure, always)

Complete Jenkinsfile Example (Java App)

```
pipeline {
    agent {
        label 'linux'  // Run on Linux agent
    }
```

```
environment {
    DOCKER_REGISTRY = 'myregistry.azurecr.io'
    IMAGE_NAME = 'myapp'
    SONAR_HOST = 'http://sonarqube:9000'
}

triggers {
    pollSCM('H/5 * * * *')  // Poll Git every 5 minutes
}

stages {
    stage('Checkout') {
        steps {
            checkout scm  // Pull code from Git
        }
    }

    stage('Build') {
        steps {
            sh 'mvn clean package -DskipTests'
        }
    }

    stage('Unit Tests') {
        steps {
            sh 'mvn test'
        }
        post {
            always {
                junit 'target/surefire-reports/*.xml'
            }
        }
    }

    stage('SonarQube Analysis') {
        steps {
            withSonarQubeEnv('SonarQube') {
                sh 'mvn sonar:sonar'
            }
        }
    }

    stage('Build Docker Image') {
        steps {
            script {
```

```
                    def image =
docker.build("${DOCKER_REGISTRY}/${IMAGE_NAME}:${BUILD_NUMBER}")
                }
            }
        }

        stage('Push to Registry') {
            steps {
                script {
                    docker.withRegistry('https://myregistry.azurecr.io',
'acr-credentials') {

docker.image("${DOCKER_REGISTRY}/${IMAGE_NAME}:${BUILD_NUMBER}").push()

docker.image("${DOCKER_REGISTRY}/${IMAGE_NAME}:${BUILD_NUMBER}").push('latest')
                    }
                }
            }
        }

        stage('Deploy to Staging') {
            steps {
                sh '''
                    kubectl set image deployment/myapp \
                        myapp=${DOCKER_REGISTRY}/${IMAGE_NAME}:${BUILD_NUMBER} \
                        -n staging
                '''
            }
        }

        stage('Integration Tests') {
            steps {
                sh './run-integration-tests.sh staging'
            }
        }

        stage('Deploy to Production') {
            when {
                branch 'main'  // Only deploy from main branch
            }
            steps {
                input message: 'Deploy to Production?', ok: 'Deploy'
                sh '''
                    kubectl set image deployment/myapp \
                        myapp=${DOCKER_REGISTRY}/${IMAGE_NAME}:${BUILD_NUMBER} \
                        -n production
                '''
```

```
            }
        }
    }

    post {
        success {
            slackSend channel: '#deployments',
                      color: 'good',
                      message: "Deployed ${IMAGE_NAME}:${BUILD_NUMBER} to
production"
        }
        failure {
            slackSend channel: '#deployments',
                      color: 'danger',
                      message: "Pipeline FAILED for ${IMAGE_NAME}:${BUILD_NUMBER}"
        }
    }
}
```

Key Jenkinsfile Concepts

- **Agent Specification**

```
agent any                    // Run on any agent
agent none                   // Don't allocate agent (manually specify per stage)
agent { label 'linux' }      // Run on agent with label 'linux'
agent { docker 'node:16' }   // Run inside Docker container
```

- **Environment Variables**

```
environment {
    PATH = "/usr/local/bin:$PATH"
    DB_URL = credentials('database-url')  // Retrieve from Jenkins credentials
}
```

- **Conditional Execution**

```
when {
    branch 'main'           // Only run on main branch
    environment name: 'ENV', value: 'prod'
    expression { return params.DEPLOY == true }
}
```

- **Parallel Stages**

```
stage('Parallel Tests') {
    parallel {
        stage('Unit Tests') {
            steps { sh 'npm run test:unit' }
        }
        stage('Lint') {
            steps { sh 'npm run lint' }
```

```
        }
        stage('Security Scan') {
            steps { sh 'npm audit' }
        }
    }
}
```

💡 **TIP:** Use parallel stages for tests that don't depend on each other. Can reduce pipeline time by 50-70%.

---

5. Jenkins Best Practices

5.1 Pipeline Organization

- **One Jenkinsfile per Repository** — Keep pipeline definition with application code
- **Use Shared Libraries** — Extract common logic (deploy functions, notifications) into reusable libraries
- **Parameterize Pipelines** — Make pipelines configurable (environment, version, deploy target)
- **Fail Fast** — Run cheap/fast checks first, expensive checks later
- **Retry Flaky Steps** — Network calls, external services can be flaky — add retry logic

5.2 Agent Management

- **Don't Run Builds on Master** — Master should only orchestrate, not execute builds
- **Use Docker Agents** — Disposable, isolated, consistent environments
- **Label Agents** — linux, windows, docker, high-cpu — route jobs to appropriate agents
- **Clean Workspace** — Always clean workspace before build to avoid stale artifacts

5.3 Security

- **Credentials Plugin** — Store all secrets in Jenkins Credentials, reference by ID
- **Role-Based Access** — Use Jenkins Role-Based Strategy plugin for access control
- **Least Privilege** — Pipeline should have minimum permissions needed
- **Audit Logs** — Enable audit trail plugin to track all actions
- **Secure Jenkins** — Enable HTTPS, keep Jenkins updated, disable anonymous access

⚠️ **WARNING:** Never use 'sh' or 'bat' to echo credentials. They'll appear in console logs. Use withCredentials() block instead.

PART 3
GITHUB ACTIONS

6. GitHub Actions

6.1 What is GitHub Actions?

GitHub Actions is GitHub's built-in CI/CD platform. It's tightly integrated with GitHub repositories. You define workflows in YAML files stored in .github/workflows/ directory. Every push, PR, or schedule can trigger a workflow.

Why Use GitHub Actions?

- **Native GitHub Integration** — No separate CI/CD tool needed, everything in one place
- **Free for Public Repos** — Unlimited minutes for open source. Private repos: 2000 min/month free
- **Huge Marketplace** — 10,000+ pre-built actions for common tasks
- **Matrix Builds** — Test across multiple OS, Node versions, etc. in parallel
- **Self-Hosted Runners** — Run on your own infrastructure if needed

6.2 GitHub Actions Concepts

Concept	Description
Workflow	Automated process defined in YAML file. Can have multiple workflows per repo.
Event	Trigger that starts a workflow (push, pull_request, schedule, workflow_dispatch)
Job	Set of steps that run on the same runner. Workflows can have multiple jobs.
Step	Individual task within a job (run command, use an action)
Action	Reusable unit of code (checkout code, setup Node.js, deploy to Azure)
Runner	Server that runs your workflows (GitHub-hosted or self-hosted)

6.3 Workflow File Structure

```yaml
# .github/workflows/ci.yml
name: CI Pipeline                    # Workflow name (shown in GitHub UI)

on:                                  # Events that trigger workflow
  push:
```

```yaml
    branches: [main, develop]       # Run on push to these branches
  pull_request:
    branches: [main]                # Run on PRs to main
  schedule:
    - cron: '0 2 * * *'            # Run daily at 2 AM UTC
  workflow_dispatch:                # Allow manual trigger

env:                                 # Environment variables for all jobs
  NODE_VERSION: '18'

jobs:                                # Define jobs
  build:                            # Job ID
    runs-on: ubuntu-latest          # Runner type

    steps:                          # Steps within job
      - name: Checkout code
        uses: actions/checkout@v3   # Use pre-built action

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: ${{ env.NODE_VERSION }}

      - name: Install dependencies
        run: npm ci                 # Run shell command

      - name: Run tests
        run: npm test
```

6.4 Complete GitHub Actions Example

```yaml
name: Complete CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  test:
    name: Test and Build
```

```yaml
  runs-on: ubuntu-latest

  strategy:
    matrix:
      node-version: [16, 18, 20]  # Test on multiple Node versions

  steps:
    - uses: actions/checkout@v3

    - name: Setup Node.js ${{ matrix.node-version }}
      uses: actions/setup-node@v3
      with:
        node-version: ${{ matrix.node-version }}
        cache: 'npm'               # Cache npm dependencies

    - name: Install dependencies
      run: npm ci

    - name: Run linter
      run: npm run lint

    - name: Run unit tests
      run: npm test -- --coverage

    - name: Upload coverage to Codecov
      uses: codecov/codecov-action@v3
      with:
        file: ./coverage/coverage-final.json

build-and-push:
  name: Build and Push Docker Image
  runs-on: ubuntu-latest
  needs: test                      # Wait for test job to complete
  if: github.event_name == 'push'  # Only on push, not PR

  permissions:
    contents: read
    packages: write

  steps:
    - uses: actions/checkout@v3

    - name: Log in to Container Registry
      uses: docker/login-action@v2
      with:
        registry: ${{ env.REGISTRY }}
```

```yaml
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}

    - name: Extract metadata
      id: meta
      uses: docker/metadata-action@v4
      with:
        images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
        tags: |
          type=ref,event=branch
          type=sha

    - name: Build and push
      uses: docker/build-push-action@v4
      with:
        context: .
        push: true
        tags: ${{ steps.meta.outputs.tags }}
        labels: ${{ steps.meta.outputs.labels }}

deploy-staging:
  name: Deploy to Staging
  runs-on: ubuntu-latest
  needs: build-and-push
  if: github.ref == 'refs/heads/develop'

  environment:
    name: staging
    url: https://staging.myapp.com

  steps:
    - name: Deploy to Azure Web App
      uses: azure/webapps-deploy@v2
      with:
        app-name: 'myapp-staging'
        publish-profile: ${{ secrets.AZURE_WEBAPP_STAGING }}
        images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:develop

deploy-production:
  name: Deploy to Production
  runs-on: ubuntu-latest
  needs: build-and-push
  if: github.ref == 'refs/heads/main'

  environment:
    name: production
```

```yaml
      url: https://myapp.com

    steps:
      - name: Deploy to Azure Web App
        uses: azure/webapps-deploy@v2
        with:
          app-name: 'myapp-production'
          publish-profile: ${{ secrets.AZURE_WEBAPP_PROD }}
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:sha-${{ github.sha }}
```

Key GitHub Actions Features

▸ 1. Matrix Strategy

Run same job across multiple configurations in parallel.

```yaml
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest, macos-latest]
    node: [16, 18, 20]
    # Creates 9 jobs (3 OS × 3 Node versions)
```

▸ 2. Reusable Workflows

Call one workflow from another (avoid duplication).

```yaml
# In .github/workflows/reusable-deploy.yml
on:
  workflow_call:
    inputs:
      environment:
        required: true
        type: string

# Call from another workflow
jobs:
  deploy:
    uses: ./.github/workflows/reusable-deploy.yml
    with:
      environment: production
```

▸ 3. Composite Actions

Create your own reusable actions.

```yaml
# .github/actions/setup-app/action.yml
name: 'Setup Application'
description: 'Install Node.js and dependencies'

runs:
  using: 'composite'
  steps:
    - uses: actions/setup-node@v3
```

```
      with:
        node-version: '18'
    - run: npm ci
      shell: bash
```

4. Secrets and Environments

```
# Access secrets
${{ secrets.API_KEY }}
${{ secrets.AZURE_CREDENTIALS }}

# Use environments for approvals
environment:
  name: production              # Requires approval before running
  url: https://myapp.com
```

💡 **TIP:** GitHub Marketplace has 10,000+ actions. Before writing custom code, search marketplace — someone probably already built it!

PART 4

AZURE DEVOPS PIPELINES

7. Azure DevOps Pipelines

7.1 What is Azure DevOps?

Azure DevOps is Microsoft's complete DevOps platform. It includes: Boards (project management), Repos (Git), Pipelines (CI/CD), Artifacts (package management), and Test Plans. Azure Pipelines is the CI/CD component.

Why Use Azure Pipelines?

- **Cloud + On-Premises** — Supports Azure, AWS, GCP, and on-premises deployments
- **Multi-Platform** — Linux, Windows, macOS agents. Build for any platform.
- **Language Agnostic** — Java, .NET, Python, Node.js, Go, Ruby, etc.
- **Free Tier** — 1800 minutes/month for private projects, unlimited for public
- **Enterprise Ready** — Compliance, security, audit logs built-in

7.2 Azure Pipeline YAML Structure

```yaml
# azure-pipelines.yml
trigger:                     # When to run pipeline
  branches:
    include:
      - main
      - develop
  paths:
    exclude:
      - docs/*              # Don't trigger for doc changes

pr:                         # Run on Pull Requests
  branches:
    include:
      - main

variables:                  # Pipeline variables
  buildConfiguration: 'Release'
  dotnetVersion: '7.x'

stages:                     # Pipeline stages
  - stage: Build
    displayName: 'Build Stage'
    jobs:
```

```
      - job: BuildJob
        displayName: 'Build and Test'
        pool:
          vmImage: 'ubuntu-latest'
        steps:
          - task: UseDotNet@2
            inputs:
              version: $(dotnetVersion)

          - task: DotNetCoreCLI@2
            displayName: 'Build'
            inputs:
              command: 'build'
              configuration: $(buildConfiguration)
```

7.3 Complete Azure Pipeline Example

```
trigger:
  branches:
    include: ['main', 'develop']

variables:
  - group: Production-Variables  # Variable group from Library
  - name: dockerRegistryServiceConnection
    value: 'MyACRConnection'
  - name: imageRepository
    value: 'myapp'
  - name: containerRegistry
    value: 'myregistry.azurecr.io'
  - name: tag
    value: '$(Build.BuildId)'

stages:
  - stage: Build
    displayName: 'Build and Test'
    jobs:
      - job: Build
        pool:
          vmImage: 'ubuntu-latest'
        steps:
          - task: NodeTool@0
            inputs:
              versionSpec: '18.x'
            displayName: 'Install Node.js'

          - script: |
```

```
              npm ci
              npm run build
            displayName: 'npm install and build'

          - task: PublishTestResults@2
            inputs:
              testResultsFormat: 'JUnit'
              testResultsFiles: '**/test-results.xml'
            displayName: 'Publish Test Results'

          - task: Docker@2
            displayName: 'Build and push image'
            inputs:
              command: buildAndPush
              repository: $(imageRepository)
              dockerfile: '$(Build.SourcesDirectory)/Dockerfile'
              containerRegistry: $(dockerRegistryServiceConnection)
              tags: |
                $(tag)
                latest

  - stage: DeployStaging
    displayName: 'Deploy to Staging'
    dependsOn: Build
    condition: and(succeeded(), eq(variables['Build.SourceBranch'],
'refs/heads/develop'))
    jobs:
      - deployment: DeployStaging
        environment: 'staging'
        pool:
          vmImage: 'ubuntu-latest'
        strategy:
          runOnce:
            deploy:
              steps:
                - task: AzureWebAppContainer@1
                  inputs:
                    azureSubscription: 'MyAzureSubscription'
                    appName: 'myapp-staging'
                    containers: '$(containerRegistry)/$(imageRepository):$(tag)'

  - stage: DeployProduction
    displayName: 'Deploy to Production'
    dependsOn: DeployStaging
    condition: and(succeeded(), eq(variables['Build.SourceBranch'],
'refs/heads/main'))
    jobs:
```

```
      - deployment: DeployProduction
        environment: 'production'  # Requires manual approval
        pool:
          vmImage: 'ubuntu-latest'
        strategy:
          runOnce:
            deploy:
              steps:
                - task: AzureWebAppContainer@1
                  inputs:
                    azureSubscription: 'MyAzureSubscription'
                    appName: 'myapp-production'
                    containers: '$(containerRegistry)/$(imageRepository):$(tag)'
```

Azure Pipelines Key Concepts

▸ Service Connections

Secure way to connect to external services. Created in Project Settings.

- Azure Resource Manager — Connect to Azure subscription
- Docker Registry — ACR, Docker Hub, private registry
- Kubernetes — Connect to AKS or other K8s clusters
- GitHub — Access GitHub repos
- NPM — Private npm registries

▸ Environments

Deployment targets with approvals, checks, and history.

```
# Create environment in Azure DevOps UI
# Then reference in pipeline
environment: 'production'   # Shows deployment history in UI
```

▸ Variable Groups

Share variables across multiple pipelines. Can link to Azure Key Vault.

```
variables:
  - group: 'Prod-KeyVault'  # Variables from Key Vault
  - name: 'customVar'
    value: 'value'
```

▸ Pipeline Templates

Reuse YAML across multiple pipelines.

```
# templates/build-template.yml
parameters:
  - name: buildConfiguration
    type: string
    default: 'Release'

steps:
  - script: dotnet build --configuration ${{ parameters.buildConfiguration }}
```

```yaml
# Use in main pipeline
steps:
  - template: templates/build-template.yml
    parameters:
      buildConfiguration: 'Debug'
```

💡 **TIP:** Azure Pipelines has 100+ built-in tasks. Browse tasks in the Azure DevOps task catalog before writing custom scripts.

Additional comprehensive content continues in your document covering:

32. Approval Gates & Environment Controls
33. Pipeline Templates & Code Reuse Patterns
34. 30+ Interview Questions (Basic to Advanced)
35. Scenario-Based Questions with Solutions
36. 15+ Hands-On Practice Assignments
37. 5 Complete CI/CD Project Ideas
38. Common Mistakes & Debugging Tips
39. Quick Revision Cheat Sheets

END OF CI/CD STUDY GUIDE

Master CI/CD pipelines. Automate everything. Ship faster, safer, better!